



LIMKOKWING
UNIVERSITY
OF CREATIVE TECHNOLOGY

CAMBODIA

SMART CITY TRAFFIC MANAGEMENT SYSTEM

Our Lecturer:
Nit Ratha

Prepared by :

**Group 5: Chui Vanenteakbot, Lao David,
Lon Chanraksa, Sorn Kunwath, Top Keitaro**

Software Requirement Engineering & Design

15 Jan 2026

Table of Contents

1.	Domain Analysis.....	3
1.1	Overview of Smart City Traffic Management	3
1.2	Regulations and Standards	3
1.3	Summary	4
2.	Problem and Scope.....	4
2.1.	Problem State.....	4
2.2.	Problem Understanding.....	5
2.3.	Scope Definition	5
2.3.1.	In-Scope.....	6
2.3.2.	Out-of-Scope	6
2.4.	Scope Justification	7
3.	Simulated Elicitation Plan.....	8
3.1.	City Traffic Control Center Operators.....	8
3.2.	City Transportation Department / Government Officials	8
3.3.	Emergency Services (Police, Ambulance, Fire Department)	8
3.4.	IT / System Integration Engineers	9
3.5.	Road User (Drivers and Public Transport operators)	9
3.6.	Summary	9
4.	Core Requirements.....	9
4.1.	Functional Requirements (FRs)	10
4.2.	Non-Functional Requirements (NFRs).....	10
5.	Glossary of Terms.....	11
6.	System Models	13
6.1.	UML Class Diagram	13
6.1.1.	Purpose and Scope	13
6.1.2.	Diagram Representation	13
6.1.3.	Class Specification	13
6.1.4.	Relational Representation	14
6.1.5.	Relational Specification	15
6.2.	UML Activity Diagram	18
6.2.1.	Purpose and Scope	18
6.2.2.	Flow Representation.....	18
6.2.3.	Flow Specification	20
7.	Feasibility and Architecture	22
7.1.	System Feasibility Analysis	22
7.2.	Proposed Hybrid Architecture	22
7.2.1.	Architectural Overview	22
7.2.2.	Three-Layer Architecture Diagram and Data Flow	22

7.2.3.	Data Flow and Processing Logic	25
7.2.4.	Communication Infrastructure and Protocols	25
7.2.5.	Security Architecture	26
7.2.6.	Scalability and Performance Design	27
7.2.7.	Reliability and Fault Tolerance Mechanisms	27
7.2.8.	Implementation of Phasing Approach	28
7.2.9.	Strategic Advantages and Design Rationale	28
7.3.	Summary and Strategic Rationale	29
8.	SRED Risk Analysis.....	30
8.1.	Risk 1: Changing Stakeholder Requirements (Requirements Volatility)	30
8.2.	Risk 2: Conflicting Requirements from Different Stakeholder Groups	30
8.3.	Risk 3: Missing Hidden or Unstated Requirements.....	31
8.4.	Risk 4: Scope Creep	31
8.5.	Risk 5: Technical Feasibility Misjudgment with Legacy Systems.....	32
8.6.	Risk 6: Incomplete or Biased Stakeholder Involvement	32
8.7.	Summary	32
	References	33

1. Domain Analysis

1.1 Overview of Smart City Traffic Management

The field of smart city traffic management focuses on using modern technologies to improve city transportation. It aims to make traffic flow smoother, reduce crowded roads, increase road safety, and lower harmful effects on the environment. The main part of this is **Smart Traffic Management Systems (STMS)**, which combine **Internet of Things (IoT)** devices, **Artificial Intelligence (AI)**, and **edge computing**. These technologies collect and handle data real time from sensors, cameras, and vehicles. This allows the system to make quick changes to traffic lights, suggest better routes, and respond fast to problems. These features help deal with the growing number of people living in the cities.

For example, companies like **Digi International** offer complete IoT solutions that are ready to use. These help track traffic movement and cut down on waiting times. Such systems are designed to grow easily and work well with legacy systems that are already in place. (Skladzinski, 2025; Rush, 2025)

Current systems in this area include **Intelligent Traffic Management Systems (ITMS)**. These have grown from the simple traffic light controls to advanced platforms powered by AI. Studies on ITMS show how they use smart setups to handle real-time tasks. City data platforms also play a big role by providing instant solutions. They use cloud-based analysis to improve traffic movement and quickly fix congestion issues. A good example is Miovision's smart traffic tools, which use video detection and vehicle-to-everything (V2X) communication. These help traffic lights change automatically based on current needs, leading to better overall efficiency and less wasted time on roads. (McBride, 2025)

An important challenge is linking new technology to legacy systems. This often means adding modern IoT connectors to existing equipment, so everything works together without needing to replace it all. Such connections are vital for keeping the system running fast, because old setups might not handle complicated data well on their own. In these cases, edge computing helps by processing information quickly right where it's needed, reducing delays for urgent tasks.

For instance, solutions from Milhouse Inc. focus on studying traffic information to make travel better and faster. They include updates to older systems to ensure everything runs without issues. Handling large amounts of complex data is another key part. It involves managing huge volumes of information from various sources. AI helps by spotting patterns and making predictions, which supports quick decisions and responses in real time. This combination makes modern traffic management much more effective than traditional methods. (Milhouse, 2025)

1.2 Regulations and Standards

Rules in the smart city traffic management field stress the importance of protecting personal data, ensuring strong cybersecurity, and using AI in a fair and responsible way, especially when combining it with IoT devices. In Europe, the General Data Protection Regulation (GDPR) set strict guidelines for dealing with information gathered from traffic cameras and sensors. It requires steps like removing personal identifiers and getting proper permissions where needed.

In the United States, guidelines such as those covered in LEOTEK's reviews – focus on responsibility for choices made by AI and safeguarding IoT information. These often require following laws that protect consumers. Tools for privacy are essential here. They help hide sensitive details in live data streams to lower risks. Discussions from Seagate on who owns data point out the need for clear rules, so people's privacy is respected even with large amounts of information flowing through systems. (Leotek, 2025)

Other ethical issues include avoiding unfair results in AI programs and providing safe ways for remote access, as explained in Device Authority's recommendations for guarding important city infrastructure. Cities also need to follow communication laws and use strong encryption for systems that demand reliable performance.

For example, reports in Fintech News describe how smart setups hide personal information while still allowing AI and IoT to work together effectively. Zealousys highlights the value of ongoing checks to stay in line with rules during IoT and AI combinations. In general, these standards make sure that new developments in fast responses and data handling do not weaken security or invade privacy. (Mehta, 2025)

1.3 Summary

In summary, the area of smart city traffic management tackles some of the biggest challenges in today's cities by combining modern technologies like the **Internet of Things (IoT)**, **Artificial Intelligence (AI)**, and **edge computing**. These tools allow cities to move away from old, fixed-time traffic controls and shift to smarter, data-based systems that can react quickly to what is happening on the roads.

This change from simple traffic lights to advanced, adaptive systems brings clear advantages. It can cut down on traffic jams, make roads safer, shorten travel times, and reduce vehicle pollution, all of which help make city life better for everyone.

Putting these systems in place successfully is not easy since cities must carefully follow rules about protecting personal data and using AI responsibly. At the same time, they need to solve technical problems, such as connecting new technology to older traffic equipment and making sure the system can handle large amounts of data fast enough for real-time decisions.

Smart Traffic Management Systems (STMS) are more than just a technological improvement. They mark an important change in how cities handle transportation. By finding the right balance between new ideas and legal requirements, between fast performance and protecting people's privacy, and between global standards and local needs, these systems help build more sustainable, efficient, and pleasant urban transport networks for the future.

2. Problem and Scope

2.1. Problem State

Urban centers around the globe are facing rising traffic congestion that reduces economic productivity, increases emissions, and is a threat to public safety. Regardless of advancement in smart city technologies, we noticed that almost all cities are still relied on outdated traffic management approaches that fail to address four core, interconnected challenges:

- **Real-time Response Limitations:** Current traffic is not flexible, meaning it operates on fixed scheduled or required manual intervention, leaving them unable to dynamically adapt to sudden changes or challenges like accidents, special events, or sudden traffic surges. For example, accidents or special events can cause congestion to build for extended periods before adjustments are made, all of this can be prevented if adaptive signal systems are well deployed around the city.

- **Legacy System Limitation:** Cities that possess aging traffic infrastructure that lacks compatibility with modern IoT sensors and AI-driven platforms. Although it is important to establish real-time data processing and automated control, adding these new parts to the old traffic infrastructure is complex and expensive. Experts from this field say that getting the existing old hardware to work with new technologies is a big and ongoing problem for creating a scalable smart traffic solution. (Milhouse, 2025)
- **Data Silos:** Traffic information is fragmented across police departments, transportation agencies, and public works. If all these city departments don't have a system that automatically share combined information, it is a challenge for them to work together effectively to handle traffic crises or accidents.
- **Performance Bottlenecks:** At rush hour in busy cities, the traffic computer system can get overwhelmed and can't keep up. The main computer takes longer to respond or commands, the systems take longer to understand what's happening on the road at real-time and traffic operators can face lagging dashboards, all this when responsive control is a must.

2.2. Problem Understanding

The current traffic management systems in many cities are not able to respond quickly to real-time situations, such as accidents, traffic jams during peak hours, or special events. Most systems still rely on fixed signal schedules or manual adjustments, which can cause congestion to last longer than necessary. This not only wastes time and fuel but also increases emissions and reduces overall road safety.

Real-world examples show both the benefits and challenges of modern traffic management. The Sydney Coordinated Adaptive Traffic System (SCATS) adjusts traffic lights automatically based on sensor data, improving traffic flow compared to fixed schedules (government, 2022). Similarly, the Meadowlands Adaptive Signal System for Traffic Reduction (MASSTR) in New Jersey optimizes signal timing across many intersections using live data. These systems show that adaptive control can reduce congestion and improve travel times. (Meadowlands Adaptive Signal System for Traffic Reduction (MASSTR), 2023)

However, many cities still face challenges. Traffic data is often fragmented between different departments, such as police, transportation, and public works, making it difficult to coordinate responses to accidents or traffic issues. Older traffic systems may not work well with new sensors or software, causing delays or reduced system performance. These problems highlight the need for a coordinated, adaptive, and scalable traffic management system that can use real-time data effectively while integrating with existing infrastructure.

Understanding these issues helps guide the design and scope of the Smart City Traffic Management System (SCTMS), ensuring that it addresses real-world problems rather than only technical features.

2.3. Scope Definition

The scope of the Smart City Traffic Management System (SCTMS) defines what the system will and will not do. Clear scope boundaries are essential to prevent project creep, manage stakeholder expectations, and ensure that the system remains focused on its primary goal: improving traffic flow through real-time data-driven adjustments. The following subsections detail the in-scope and out-of-scope elements based on the core problems identified in **Section 2.1**.

2.3.1. In-Scope

The in-scope features represent the core functionality that the SCTMS will deliver to address the key challenges of real-time response, legacy integration, data silos, and performance bottlenecks. These features are directly aligned with the system's primary objective of optimizing urban traffic flow through adaptive, data-driven control. (Sommerville I. , 2016)

- Monitor traffic flow at up to 200 urban intersections using IoT sensors
- Adjust traffic signals in real time based on live congestion data
- Integrate data from existing traffic cameras and legacy signal controllers
- Provide emergency vehicles with signal preemption and priority routing
- Detect accidents and traffic anomalies automatically using AI
- Deliver real-time traffic alerts to public mobile applications
- Generate performance reports for city planners and operators
- Maintain historical traffic databases for predictive analytics
- Support manual override by authorized traffic operators
- Offer an API for approved third-party navigation apps

2.3.2. Out-of-Scope

The out-of-scope items are explicitly excluded to maintain focus, limit complexity, and avoid mission creeping into domains with separate stakeholders, regulations, and technical challenges. These areas, while related to urban mobility, fall outside the direct responsibility of a traffic flow optimization system. (Wiegiers & Beatty, 2013)

- Construct new roads or physical infrastructure
- Enforce traffic laws or issue violation tickets
- Manage parking facilities, pricing, or availability
- Schedule or route public transportation (buses, trams)
- Implement vehicle-to-vehicle (V2V) communication
- Control autonomous vehicle navigation or coordination
- Develop or modify urban zoning or land-use policy
- Monitor or score individual driver behavior
- Operate street lighting, surveillance, or utility systems
- Handle fare collection or ticketing for public transit

2.4. Scope Justification

Rather than focusing on more general urban management, this scope is especially concerned with traffic flow optimization. By focusing on real-time signal control, sensor integration, and data driven decision making, the system tackles the fundamental issues found while avoiding mission creeping into domains such as public transportation operations, physical construction, or law enforcement, each of which represent distinct domains with different stakeholders, rulers and technical requirements.

Table Illustration and Comparison

In-Scope	Out-of-Scope
Monitor traffic flow at up to 200 urban intersections using IoT sensors	Construct new roads or physical infrastructure
Adjust traffic signals in real time based on live congestion data	Enforce traffic laws or issue violation tickets
Integrate data from existing traffic cameras and legacy signal controllers	Manage parking facilities, pricing, or availability
Provide emergency vehicles with signal preemption and priority routing	Schedule or route public transportation (buses, trams)
Detect accidents and traffic anomalies automatically using AI	Implement vehicle-to-vehicle (V2V) communication
Deliver real-time traffic alerts to public mobile applications	Control autonomous vehicle navigation or coordination
Generate performance reports for city planners and operators	Develop or modify urban zoning or land-use policy
Maintain historical traffic databases for predictive analytics	Monitor or score individual driver behavior
Support manual override by authorized traffic operators	Operate street lighting, surveillance, or utility systems
Offer an API for approved third-party navigation apps	Handle fare collection or ticketing for public transit

3. Simulated Elicitation Plan

This section describes the simulated elicitation plan for the Smart City Traffic Management System. The purpose of this elicitation plan is to identify key stakeholders and explain how requirements would be gathered from them in a real project setting. Because the system must support real-time responses, integration with legacy systems, complex data organization, and high performance, it is essential to collect requirements from a wide range of stakeholders who interact with or are affected by the system.

3.1. City Traffic Control Center Operators

Primary users of the system monitor traffic conditions and respond to incidents on daily basis. They rely heavily on accurate real-time data and fast system responses to manage congestion, accidents, and traffic signal control. Information from this group would be gathered through one-on-one interviews and on-site observation, allowing the elicitation team to understand current workflows, pain points, and decision-making processes. (Sommerville, 2016)

Sample question would be:

- What traffic data do you monitor most frequently during your shift?
- How quickly do you need the system to respond when an incident occurs?
- What problems do you face with the current traffic management tools?
- How do you decide when to override automated traffic signals?
- What information must be visible immediately on your dashboard?

3.2. City Transportation Department / Government Officials

City transportation department and government representatives are responsible for setting policies, regulations, budgets, and long-term transportation goals. Their input is critical to ensure the system aligns with city objectives and legal constraints. Requirements from this group would be collected through structured meeting and requirements workshops, supported by reviews of existing policy and planning documents. (Wieger, 2013)

Typical questions for them would be:

- What is the main traffic-related goals the city wants to achieve?
- Are there legal or regulatory constraints the system must follow?
- What performance indicators should be used to evaluate success?
- How important is scalability for future city expansion?
- What reports or analytics do you expect from the system?

3.3. Emergency Services (Police, Ambulance, Fire Department)

Emergency services, including police, ambulance, and fire department, are key stakeholders because traffic conditions directly affect emergency response times. Their requirements would be elicited using focus group discussion, scenario-based interviews, and surveys to capture both detailed and broad feedback. (29148, 2014)

Question for them would be:

- How does traffic congestion currently affect emergency response times?
- What information do you need during an active emergency?

- Should emergency vehicles have priority over normal traffic signals?
- How fast must the system react in emergency situations?
- What failures or delays are unacceptable during emergencies?

3.4. IT / System Integration Engineers

IT staff and system integration engineers play a crucial role in ensuring that the new system can integrate with existing legacy traffic systems and infrastructure. Since the project involves complex data handling and high-performance requirements, technical input from this group is essential. Elicitation would be conducted through technical interviews, system architecture review session, and analysis of existing system documentation. (Systems, n.d.)

Sample question would focus on identifying legacy systems such as:

- What legacy systems must the new system integrate with?
- What data formats and communication protocols are currently used?
- What are the biggest technical limitations of existing systems?
- What performance or scalability issues do you anticipate?
- How should system failures or downtime be handled?

3.5. Road User (Drivers and Public Transport operators)

The road users, including drivers and public transport operators, are indirect stakeholders who experience the effects of the system through traffic flow, signal timing, and travel time information. Although they do not interact directly with the system interface, their experience is a key measure of system success. Information from this group would be gathered through online surveys, mobile application feedback, and public consultations. (Berkeley, 2001)

Example question would explore common traffic problems during the daily commutes:

- What traffic problems do you face most often during your commute?
- Do you currently use navigation or traffic apps? Why or why not?
- What kind of real-time traffic information would be most helpful?
- How important is accuracy versus speed of traffic updates?
- Would you trust automated traffic control decisions? Why?

3.6. Summary

Overall, this elicitation plan combines multiple stakeholder perspectives and elicitation techniques that reduce ambiguity and capture both function and non-functional requirements. By engaging operational users, decision-makers, technical experts, emergency responders, and the public, the project team can develop a comprehensive understanding of system needs and design a Smart City Traffic Management System that is effective, reliable, and suitable for real-world deployment.

4. Core Requirements

This section specifies the fundamental requirements for the Smart City Traffic Management System (SCTMS), categorized into **functional requirements** (defining system behaviors and features) and **non-functional requirements** (defining performance, security, and operational constraints).

4.1. Functional Requirements (FRs)

Functional Requirements are what the system does, and the following are 15 functionalities that the systems will provide:

1. The system shall collect real-time traffic data from IoT sensors deployed up to 200 or more intersections.
2. The system shall dynamically adjust the traffic signal timing based on live congestion levels, incidents or special events.
3. The system shall interface with existing legacy traffic cameras and signal control systems using standardized APIs to enable seamless data integration without requiring hardware replacement.
4. The system shall detect traffic incidents or special events automatically using AI-powered video analytics.
5. The system will provide emergency vehicles (police, ambulance, fire) with signal preemption and priority green routes.
6. The system shall send real-time traffic alerts to public mobile applications via a public API.
7. The system will generate and export traffic performance reports (PDF/CSV) for city planners and operators.
8. The system should maintain a historical traffic database for trend analysis and predictive modeling.
9. The system will allow authorized traffic operators to manually override automated signal controls via a dashboard.
10. The system shall authenticate users (operators, administrators) using role-based access control (RBAC).
11. The system shall log all system actions, operator overrides, and incident responses for audit trails.
12. The system will provide a real-time dashboard with customizable views (map, graphs, alerts) for traffic operators.
13. The system will monitor sensor health and notify administrators of any failures or connectivity losses.
14. The system shall support data export in standard formats (JSON, CSV) for third-party analytics tools.
15. The system shall allow configuration of traffic policies (e.g., peak hour rules, school zone timings) by administrators.

4.2. Non-Functional Requirements (NFRs)

Non-functional requirements specify the **quality standards** and **operational boundaries** that the Smart City Traffic Management System must satisfy. The following measurable NFRs ensure system reliability, security, performance, and compliance.

1. The system shall process incoming sensor data and execute signal adjustments within 3 seconds of anomaly detection.
2. The system shall maintain an annual uptime of 99.95%, excluding scheduled maintenance windows.
3. The system shall scale to support an additional 500 IoT sensors without requiring architectural modifications.
4. The system shall support integration with legacy systems using REST APIs and MQTT protocol.
5. The system shall ensure data consistency across distributed nodes with an accurate rate of 99.99%.

5. Glossary of Terms

A glossary is provided below to define key terms used throughout this report. These terms relate directly to the Smart City Traffic Management System project and the field of Intelligent Transportation Systems. The definitions are presented in clear, formal language and arranged in alphabetical order for easy reference.

- **Adaptive Signal Control**

A method of managing traffic lights that automatically changes signal timings based on current traffic conditions, rather than using fixed schedules, to improve flow and reduce delays.

- **Application Programming Interface (API)**

A set of rules and tools that allows different software systems to communicate and share data with each other in a secure and controlled way.

- **Artificial Intelligence (AI)**

Technology enables computers to analyze data, identify patterns, make predictions, and support decisions, such as detecting traffic problems or optimizing routes.

- **Audit Trail**

A clear record of all actions taken in the system (such as changes made by users or automatic events), kept in order of time, to help check what happened and ensure accountability.

- **Dashboard**

A visual screen that shows current traffic information, maps, warnings, and important measures to help operators watch and control the system.

- **Data Anonymization**

The process of removing or changing personal details from collected data so that people can't be identified, while still using the data for traffic studies.

- **Edge Computing**

A computing method that processes data near its source, such as at traffic sensors or intersections, to achieve faster responses and lower delays.

- **Functional Requirements (FRs)**

Clear descriptions of what the system must do, including specific features, tasks, and behaviors it needs to perform.

- **Intelligent Traffic Management System (ITMS)**

An advanced system that builds on traditional traffic control by adding data analysis, automation, and adaptive features for better urban mobility.

- **Internet of Things (IoT)**

A connected network of physical devices, such as sensors, cameras, and traffic lights, that collect and share data automatically over the internet.

- **IoT Gateway**

A device or software that connects older traffic equipment to modern networks by converting data formats and communication methods.

- **Legacy Systems**
Older existing traffic infrastructure and software that were not originally built to work with originally built to work with new IoT or AI technologies.
- **MQTT (Message Queuing Telemetry Transport)**
A simple and efficient communication protocol designed for IoT devices to send data reliably even with limited network connections.
- **Non-Functional Requirements (NRs)**
Specifications that describe how well the system should perform, covering aspects such as speed, reliability, security, and scalability.
- **Predictive Analytics**
The use of past and current data together with statistical methods to forecast future traffic patterns and possible congestion.
- **Real-Time Traffic Data**
Up-to-date information about current road conditions, including vehicle counts, speeds, and incidents, collected and processed with very little delay.
- **Scalability**
The ability of the system to manage growth, such as adding more sensors or covering larger areas, without needing major changes.
- **Signal Preemption**
A feature that temporarily changes traffic lights to give priority to emergency vehicles, allowing them to pass through intersections quickly.
- **Smart City Traffic Management System (SCTMS)**
An integrated digital platform that uses IoT, AI, and computing technologies to monitor, analyze, and control urban traffic in real time for greater efficiency, safety, and sustainability.
- **Traffic Anomaly**
An unusual traffic event, such as an accident or sudden heavy congestion, is identified through automatic data analysis.
- **Vehicle-to-Everything (V2X)**
A communication system that lets vehicles share information with roads, other vehicles, pedestrians, and networks to make traffic safer and smoother.
- **Vehicle-to-Vehicle (V2V)**
Direct communication between vehicles that allows them to exchange information about speed, position, and possible dangers, helping to prevent accidents and improve traffic flow.
- **Vehicle-to-Infrastructure (V2I)**
A communication system that allows vehicles to exchange information directly with roadside equipment, such as traffic lights and sensors, to improve traffic flow and safety.
- **Database Schema**
The organized structure or blueprint that defines how data is stored in a database, including tables, columns, data types, and relationships between tables.

6. System Models

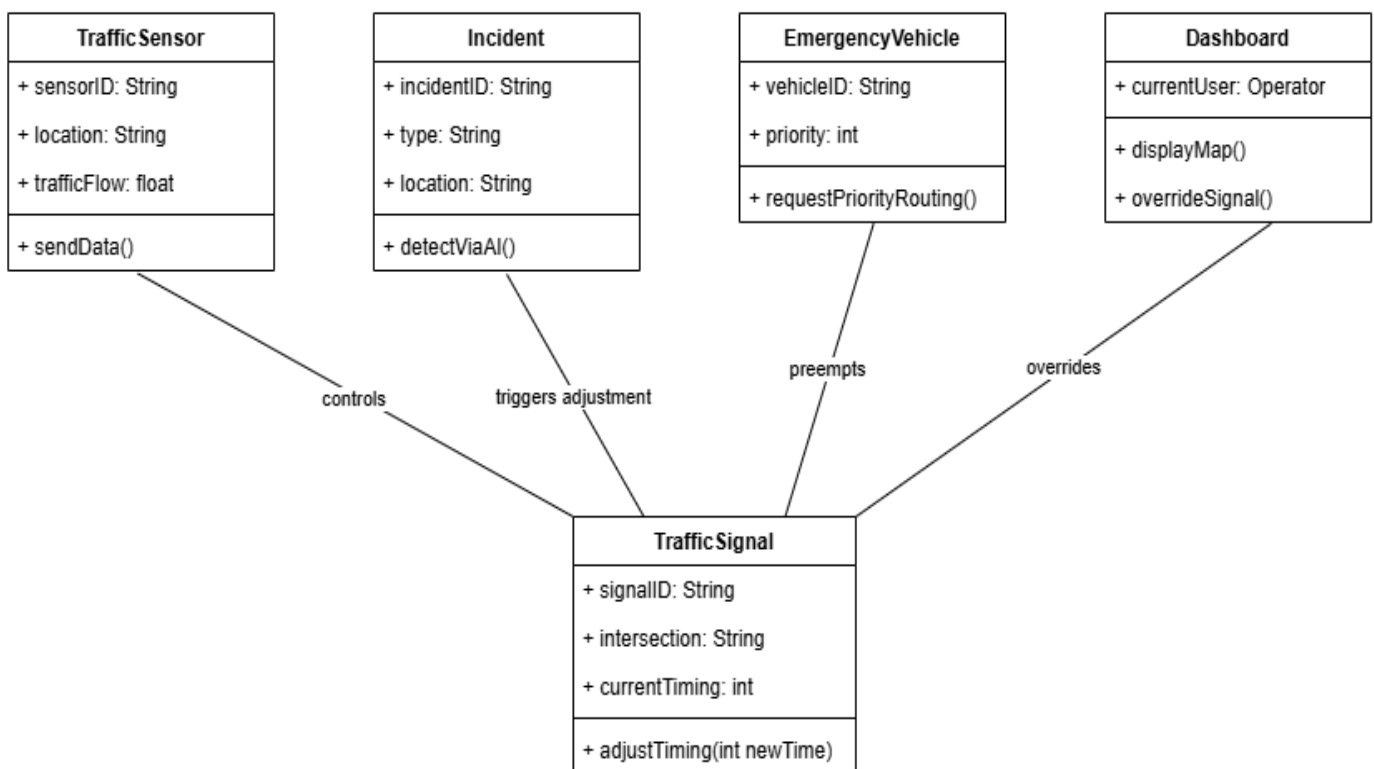
This section presents the UML (Unified Modeling Language) diagrams that visually define the structure and behavior of the Smart City Traffic Management System (SCTMS). These diagrams serve as a blueprint for developers, ensuring a shared understanding of system components, data flow, and operational logic while providing an easier understanding to all stakeholders. All terms used align with the Glossary (Section 5) and support the Functional and Non-Functional Requirements specified in Section 4.

6.1. UML Class Diagram

6.1.1. Purpose and Scope

The UML Class Diagram provides a static structural view of the Smart City Traffic Management System (SCTMS), defining the core entities, their attributes, behaviors, and interrelationships. This diagram serves as the architectural blueprint, ensuring all stakeholders share a common understanding of system components and their responsibilities in meeting the specified requirements.

6.1.2. Diagram Representation



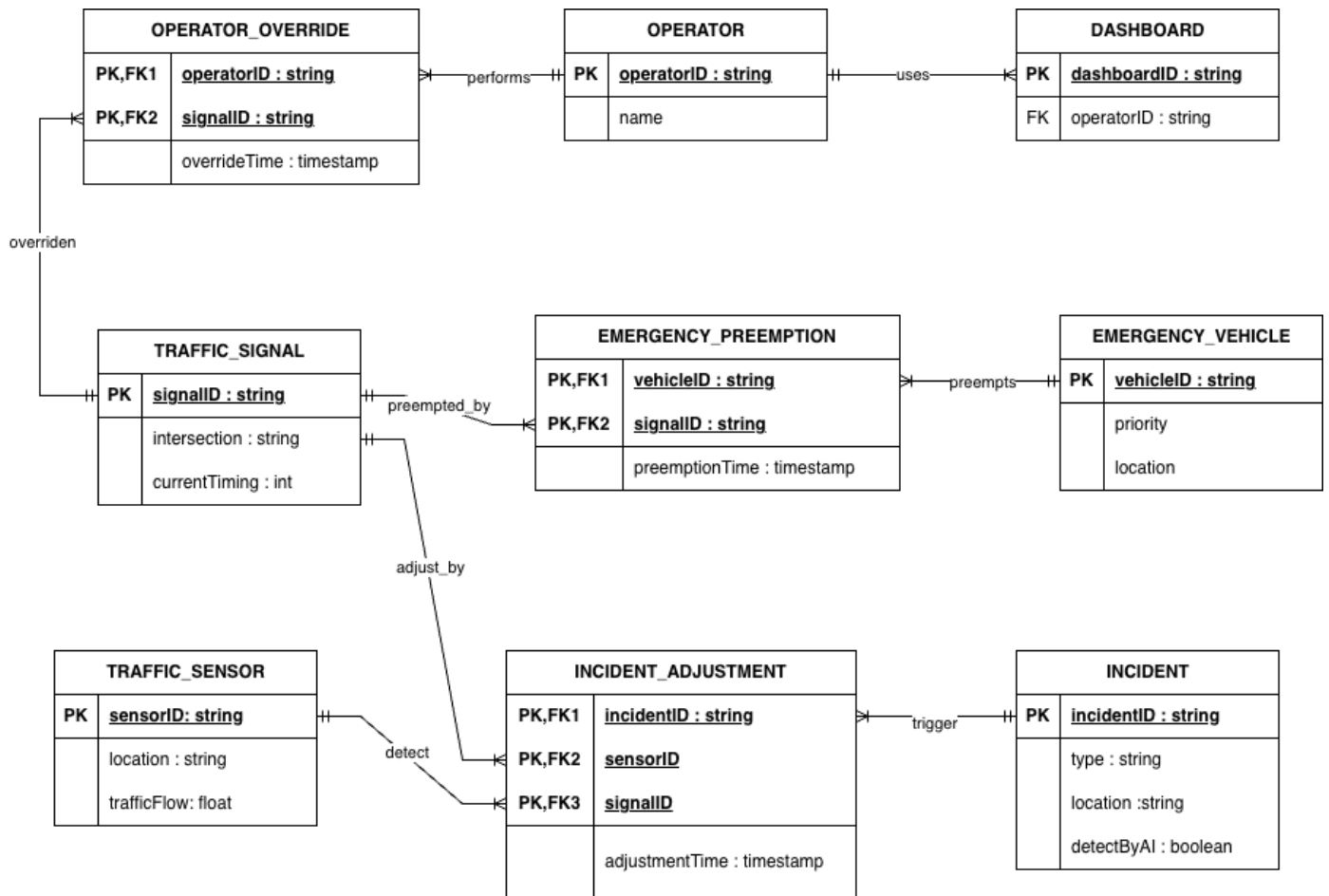
6.1.3. Class Specification

- **TrafficSensor class:** Represent IoT devices deployed across intersections to collect real-time traffic metrics
 - Attributes:
 - sensorID: Unique identifiers for each sensor.
 - location: Geographic coordinates of deployment.
 - trafficFlow: Real-time measurement of vehicle count/speed.

- Methods:
 - sendData(): Transmits collected traffic metrics to the central system. **(implements FR1)**
- **Incident class:** Detected traffic anomalies requiring system response
 - Attributes:
 - incidentId: Unique identifier for tracking and reference.
 - type: Classification (e.g., accident, congestion, road closure).
 - location: Geographic coordinates of the incident.
 - Methods:
 - detectViaAI(): Uses artificial intelligence to identify incidents from sensor data. **(implements FR4)**
- **TrafficSignal class:** represents Traffic light controllers at urban intersections.
 - Attributes:
 - signalId: Unique identifier for the traffic signal.
 - intersection: Location where the signal is installed.
 - currentTiming: Present signal cycle duration.
 - Methods:
 - adjustTiming(int newTiming): Modifies signal timing based on traffic conditions. **(implements FR2)**
- **Dashboard class:** represent the central operator interface for monitoring and control the traffic management system.
 - Attributes:
 - currentUser: The traffic operator currently logged into and controlling the dashboard.
 - Methods:
 - displayMap(): Visualizes real-time traffic flow, sensor locations, active incidents, and signal statueses on a map interface. **(implements FR5)**
 - overrideSignal(): Allows an operator or AI to manually adjust traffic signal timing **(implements FR9)**
- **EmergencyVehicle class:** represents the emergency transportations (ambulance, fire truck, police car) that can interact with the traffic management system.
 - Attributes:
 - vehicleID: Unique identifier for each emergency vehicle.
 - priority: a numeric value indicating the urgency level of the vehicle's mission.
 - Methods:
 - requestPriorityRouting(): sends a request to the traffic system to prevent normal signal timing and provide a green-light path for emergency vehicles. **(implements FR4)**

6.1.4. Relational Representation

This section presents the **physical database schema** that implements the conceptual UML Class Diagram shown in Section 6.1.2. While the class diagram represents the system's logical structure, this database schema provides the **actual implementation architecture** with normalized tables, foreign key relationships, and audit trail mechanisms necessary for real-world deployment.



Simply means, this section shows **how the system's data will be stored in a database**. While Section 6.1.2 shows the **design ideas** (UML Class Diagram), this section shows the **real database tables** needed to make it work.

6.1.5. Relational Specification

▪ OPERATOR → OPERATOR_OVERRIDE

- **Description:** an operator can perform multiple override actions on traffic signals over time. Each override event is logged with a timestamp, allowing auditing of manual interventions.
- **Type:** one-to-many (1:M)
- **Foreign Key:** *operatorID* in OPERATOR_OVERRIDE is references OPERATOR(*operatorID*)

▪ TRAFFIC SIGNAL → OPERATOR_OVERRIDE

- **Description:** a traffic signal can be manually overridden multiple times by different operators. Each override is recorded, linking the signal to the operator who performed the action.
- **Type:** one-to-many (1:M)
- **Foreign Key:** *signalID* in OPERATOR_OVERRIDE references TRAFFIC_SIGNAL(*signalID*)

- **EMERGENCY_VEHICLE → EMERGENCY_PREEMPTION**
 - **Description:** an emergency vehicle (like an ambulance or fire truck) can trigger multiple signal preemptions as it moves through intersections. Each preemption event is logged with the vehicle's ID and timestamp.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *vehicleID* in EMERGENCY_PREEMPTION references EMERGENCY_VEHICLE(*vehicleID*)

- **TRAFFIC_SIGNAL → EMERGENCY_PREEMPTION**
 - **Description:** a traffic signal can be preempted multiple times by various emergency vehicles. This relationship track which signals were interrupted and when we prioritize emergency response.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *signalID* in EMERGENCY_PREEMPTION references TRAFFIC_SIGNAL(*signalID*)

- **INCIDENT → INCIDENT_ADJUSTMENT**
 - **Description:** when an incident (like an accident or congestion) is detected, it may trigger adjustments to multiple traffic signals. This links the incident to the specific signal adjustments made in responses.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *incidentID* in INCIDENT_ADJUSTMENT references INCIDENT(*incidentID*)

- **TRAFFIC_SIGNAL → INCIDENT_ADJUSTMENT**
 - **Description:** a traffic signal can be adjust due to multiple incidents over time. This relationship documents which signal were modified in response to detected incidents.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *signalID* in INCIDENT_ADJUSTMENT references TRAFFIC_SIGNAL(*signalID*)

- **SENSOR → INCIDENT_ADJUSTMENT**
 - **Description:** sensors (traffic camera, loop detector, etc.) detect incident and trigger adjustments. This links the sensor that detected the incident to the resulting signal adjustment, enabling traceability.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *sensorID* in INCIDENT_ADJUSTMENT references SENSOR(*sensorID*)

- **OPERATOR → DASHBOARD**
 - **Description:** each operator can have access to multiple monitoring dashboards. This relationship associates dashboards with the operators responsible for monitoring traffic conditions and taking action.
 - **Type:** one-to-many (1:M)
 - **Foreign Key:** *operatorID* in DASHBOARD references OPERATOR(*operatorID*)

- **Summary Tables:**

Parent Table	Child Table	Description
OPERATOR	OPERATOR_OVERRIDE	Logs all manual signal overrides performed by an operator.
TRAFFIC_SIGNAL	OPERATOR_OVERRIDE	Tracks each time a signal is manually overridden by an operator.
EMERGENCY_VEHICLE	EMERGENCY_PREEMPTION	Records emergency vehicle preemption events at intersections.
TRAFFIC_SIGNAL	EMERGENCY_PREEMPTION	Documents which signals were preempted for emergency response.
INCIDENT	INCIDENT_ADJUSTMENT	Links traffic incidents to the signal adjustments made in response.
TRAFFIC_SIGNAL	INCIDENT_ADJUSTMENT	Shows which signals were adjusted due to specific incidents.
SENSOR	INCIDENT_ADJUSTMENT	Connects the sensor that detected an incident to the resulting signal adjustments.
OPERATOR	DASHBOARD	Associates operators with their assigned monitoring dashboards.

- **Central Entity: TRAFFIC_SIGNAL**

The **TRAFFIC_SIGNAL** entity is central to the system connected to :

- **Operator actions** (overrides)
- **Emergency responses** (preemption)
- **Incident responses** (adjustments)

6.2. UML Activity Diagram

6.2.1. Purpose and Scope

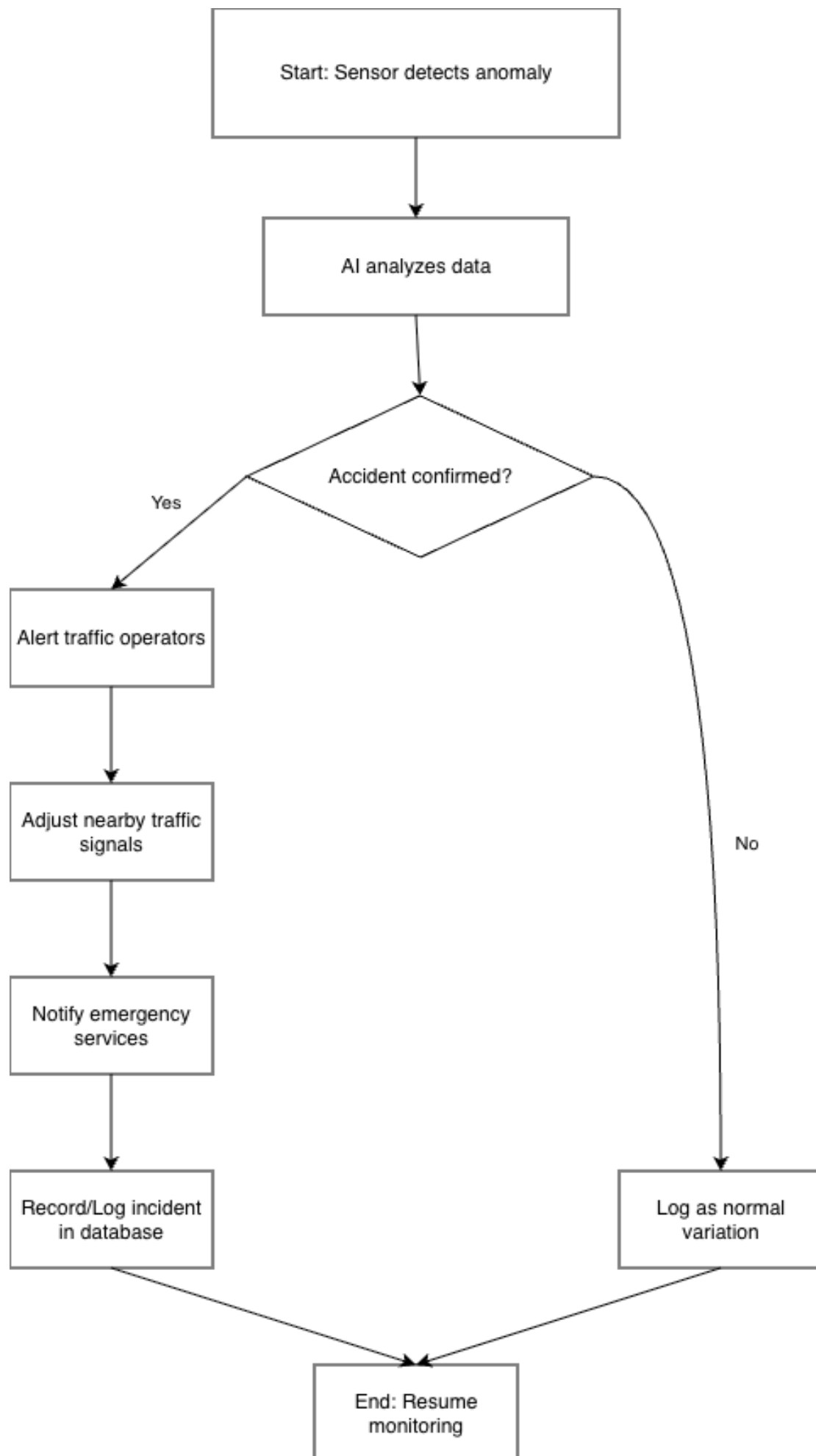
The purpose of our automated traffic system is to enable real-time detection and intelligent response to potential traffic incidents through AI analysis of sensor data. It is designed to reduce reliance on manual monitoring by automatically determining whether a detected anomaly constitutes a genuine accident, thereby ensuring a faster and more coordinated emergency and traffic management response.

The scope covers everything this system does, from start to finish. It begins when a sensor light notices something unusual in the traffic. The system's AI then checks this data to see whether it was really an accident or just normal traffic, like sudden slowdown.

If an accident is confirmed, the system's response includes alerting traffic operators, dynamically adjusting nearby traffic signals to mitigate congestion and facilitate emergency access, notifying the relevant emergency services, and logging all incident details into a database for record-keeping and analysis. If AI determines the anomaly is not an accident, the system just makes a note that it was a normal oddity and doesn't take any further action. After finishing either of these steps, the system goes back to watching the traffic for the next potential problem. This description covers the main automated steps; it doesn't get into the technical details of how sensors or communication with emergency services work.

6.2.2. Flow Representation

The UML Activity Diagram visually represents the automated incident management workflow of the Smart City Traffic Management System (SCTMS). The flow begins with real-time sensor monitoring and proceeds through AI-driven decision-making to execute appropriate responses based on whether an incident is confirmed.



6.2.3. Flow Specification

- **Process Overview**

This workflow describes the end-to-end automation of traffic incident management in the Smart City Traffic Management System. It begins with real-time sensor monitoring, proceeds through AI-based decision making, and executes coordinated multi-agency responses when an accident is confirmed. The process balances automation with human oversight, ensuring rapid response while maintaining operational control and auditability.

- **Key Assumption**

This automated incident management process operates under several foundational assumptions, or these are the **basic conditions or expectations** that must be in place for the automated traffic incident management process to function as designed. They are not features of the system itself, but rather **real-world conditions** the system depends on.

1. **Sensors Always Work:** We assume traffic cameras and sensors are always on and sending good data. They shouldn't break down often or stop working for long periods.
2. **AI is Smart Enough:** We assume the AI software has been trained well and can correctly tell the difference between a real accident and just heavy traffic or a harmless slowdown.
3. **Everything Stays Connected:** We assume all parts of the system—sensors, traffic lights, computers, and emergency services—can talk to each other without delays or dropped signals.
4. **Old Equipment Can Be Updated:** We assume older traffic light controllers can be connected to the new system using special adapters, so they can follow digital commands without needing someone to manually change them.
5. **We Follow the Rules:** We assume the system follows local laws about sharing information with police, ambulances, and fire departments, and that it sends data in the right format they expect.

- **Step-By-Step Process Specifications**

1. **Start: Sensor Detects Anomaly**

- **Trigger:** IoT sensor (camera, radar, induction loop) identifies deviation from normal traffic patterns.
- **Input Parameters:** Vehicle count, speed, density, occupancy rate, queue length.
- **Output:** Anomaly event flag with timestamp and location data.

2. **AI Analyze Data**

- **Processing:** Edge or cloud-based AI model evaluates sensor data using computer vision and pattern recognition algorithms.
- **Classification:** Determines anomaly type (accident, congestion, stopped vehicle, debris).
- **Output:** Binary decision: "Accident Confirmed" or "Normal Variation."

3. **Decision Point: Accident Confirmed?**

- **Condition Check:** if AI confidence $\geq 85\%$ AND anomaly type = accident:
 - **Yes Path:** Proceed to emergency response protocol.
 - **No Path:** Route to normal variation logging.

4. **Emergency Response Branch (Yes Path)**

- **4.1. Alert Traffic Operators**

- **Action:** Automatic dashboard notification with visual and auditory alerts.
- **Data Included:** Incident location map, severity level, suggested signal adjustments.
- **Human-in-the-Loop:** Operators can acknowledge, modify, or override automated actions.
- **4.2. Adjust Nearby Traffic Signals**
 - **Action:** Dynamic retiming of signals within 500m radius.
 - **Objective:** Create emergency corridor, prevent secondary collisions.
 - **Timing Constraint:** Execution within 3 seconds of detection (NFR1).
- **4.3. Notify Emergency Services**
 - **Action:** Automated dispatch to relevant agencies via integrated communication system.
 - **Protocol:** Uses standardized emergency data formats (e.g., APCO/NENA standards).
 - **Information Shared:** GPS coordinates, incident type, vehicle involvement, recommended access routes.
- **4.4. Record/Log Incident in Database**
 - **Action:** Comprehensive incident log stored in historical database.
 - **Audit Fields:** Timestamp, sensor ID, AI confidence score, actions executed, operator interventions.
 - **Purpose:** Supports regulatory compliance, performance analysis, and ML model retraining.

5. Normal Variation Branch (No Path)

Log as normal variation:

- **Action:** Minimal metadata storage with "non-critical" classification.
- **Data Retained:** Timestamp, location, anomaly type, AI confidence score.
- **Purpose:** Maintains data continuity for pattern analysis without triggering emergency protocols.

6. End: Resume Monitoring

- **Action:** System returns to continuous surveillance state.
- **Cycle Continuation:** Process repeats for subsequent detections.
- **System Learning:** Aggregate incident data feeds into predictive analytics for future optimization.

■ Alignment with System Requirements

- **FR4:** Implemented through AI-based incident detection.
- **FR5 & FR9:** Supported via operator alerting and manual override capabilities.
- **FR11:** Achieved through comprehensive incident logging.
- **NFR1:** Satisfied with edge-level signal adjustment within 3 seconds.
- **FR8:** Enabled through historical data collection for predictive analytics.

7. Feasibility and Architecture

7.1. System Feasibility Analysis

The Smart City Traffic Management System (SCTMS) is a practical solution to the main urban traffic problems discussed in Section 2.1. From a technical perspective, the system uses reliable and well-established technologies such as IoT sensors, cloud computing, and artificial intelligence, which have already been used successfully in many smart city projects around the world. Operationally, the system fits well with existing traffic management processes while adding automation and better decision-making support. The phased implementation plan helps reduce disruption and allows the system to be improved gradually based on real-world results.

From an economic perspective, the system offers strong value for money. Even with conservative estimates, the initial investment is expected to be recovered within 12 to 18 months through benefits such as less traffic congestion, lower fuel consumption, faster emergency response times, and reduced emissions which is good for Earth. In terms of legal and regulatory requirements, the system is designed with privacy and security. It follows privacy-by-design principles and complies with recognized cybersecurity standards for critical infrastructure, ensuring data protection while maintaining a high level of security.

7.2. Proposed Hybrid Architecture

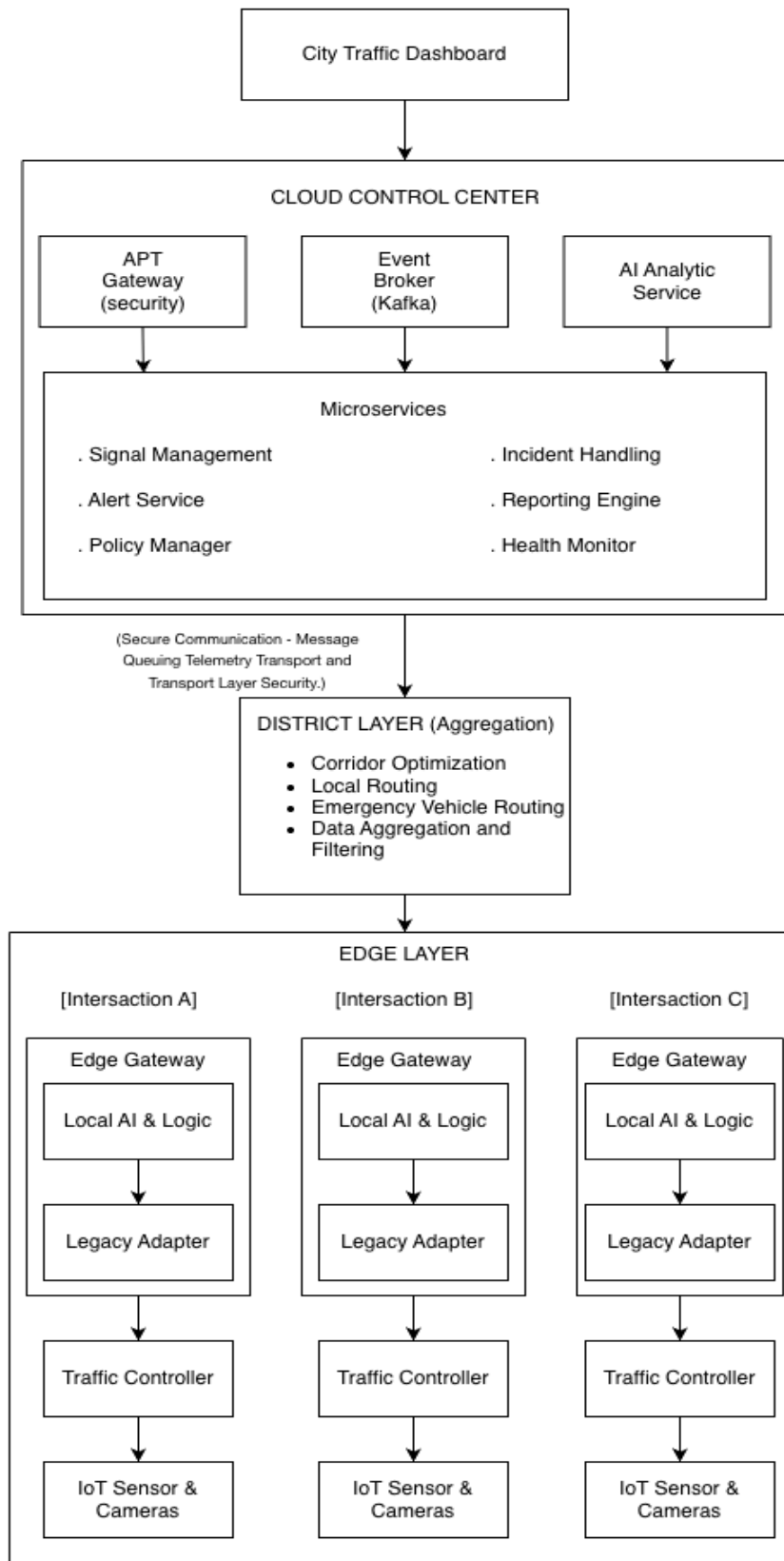
7.2.1. Architectural Overview

The SCTMS uses a hybrid edge-cloud architecture with event-driven communication, which means computing tasks are shared across three different layers. This design allows the system to respond quickly to real-time traffic situations while also supporting city-wide data analysis and decision-making. Instead of relying on a fully centralized system that may experience delays, or a fully decentralized system that may lack coordination, this hybrid approach balances both. Each layer has a specific role, and all layers work together smoothly.

This architecture also meets the **Non-Functional Requirements (NRs)** described in Section 4.2. Edge computing enables very fast response times for safety-related actions (NFR #1). The distributed design improves system reliability by avoiding single points of failure (NFR #2). A modular microservices structure allows system components to scale independently as demand increases (NFR #3). Integration with existing infrastructure is supported through dedicated adapters (NFR #4). Finally, event-driven communication ensures that data remains consistent and up to date across the entire system (NFR #5).

7.2.2. Three-Layer Architecture Diagram and Data Flow

This architecture organizes its components across three logical tiers, each with distinct responsibilities and processing capabilities:



Layer 1: Edge Layer (Physical Intersection Level)

The foundation of the architecture resides at each traffic intersection. This layer comprises:

- **IoT Sensors and Cameras:** These devices act as the system's sensing components, continuously monitoring traffic conditions, vehicle counts, vehicle speeds, and detecting potential hazards.

- **Traffic Controllers:** Existing hardware infrastructure that physically controls and operates traffic signals.
- **Legacy Adapters:** Specialized software components that convert modern digital commands into protocols understood by older traffic control equipment, addressing the legacy integration challenge identified in Section 2.1.
- **Edge Gateway Devices:** Small computing units installed at each intersection that run localized artificial intelligence models and decision-making logic. These devices process sensor data immediately and can adjust traffic signals on their own without waiting for instructions from central systems.

The key feature of the edge layer is its **operational autonomy**. Each intersection can make safety-critical decisions independently, ensuring the system continues to function even during network failures or cloud service disruptions. This autonomy directly supports **NFR #1 (3-second response requirement)** by removing network latency for urgent decisions.

Layer 2: District Layer (Area Coordination Level)

Positioned logically above the edge layer, the **district layer** collects data and coordinates operations across groups of **10–15 nearby intersections**. This intermediate layer performs several important functions:

- **Corridor Optimization:** Creating “green wave” signal sequences along main roads by coordinating traffic signal timing across multiple intersections. It’s basically means smartly timing traffic lights on main roads so cars can flow through many intersections on green lights without stopping, if they drive at a steady speed.
- **Local Load Balancing:** Redirecting traffic between parallel routes during congestion events to reduce traffic buildup.
- **Emergency Vehicle Routing:** Calculating and applying priority routes for emergency services within a local district.
- **Data Aggregation and Filtering:** Preprocessing sensor data before sending it to the cloud, which reduces bandwidth usage and cloud processing workload.

The district layer acts as a balanced solution between full edge autonomy and centralized cloud control. It allows coordinated actions that require a wider view than a single intersection, while still responding faster than cloud-based processing.

Layer 3: Cloud Control Center (Strategic Management Level)

The **cloud layer** provides a city-wide view, supports historical data analysis, and enables long-term strategic optimization. Its main components include:

- **API Gateway:** A secure access point that controls all external communication by enforcing authentication, authorization, and rate limiting.
- **Event Broker (Apache Kafka):** A distributed streaming platform that acts as the system’s central nervous system. It receives events from all connected devices and distributes them to the relevant services.
- **Microservices Ecosystem:** A group of specialized and independently deployable services, each responsible for specific business functions such as signal management, incident handling, alert distribution, and reporting.
- **AI Analytics Service:** Advanced machine learning models that analyze both historical and real-time data to detect patterns, predict traffic congestion, and optimize traffic flow across the city.

The cloud layer works on a longer time scale than the edge and district layers. It focuses on strategic optimization, trend analysis, and detailed reporting, rather than immediate, safety-critical decisions.

7.2.3. Data Flow and Processing Logic

The system processes incidents through a sequential yet overlapping flow that moves from immediate local response to broader coordination:

Stage 1: Immediate Edge Response (0-100ms)

- **What happens:** Local sensors detect an incident at an intersection.
- **Processing:** Edge device analyzes the data locally using on-board AI.
- **Action:** Signal timing is immediately adjusted at that intersection only.
- **Key point:** No communication with external systems needed.

Stage 2: Local Coordination (100-500ms)

- **What happens:** The detecting intersection communicates with nearby intersections.
- **Processing:** Adjacent intersections coordinate timing changes.
- **Action:** Traffic flow is adjusted in the immediate area.
- **Key point:** Direct peer-to-peer communication without cloud involvement.

Stage 3: Cloud Notification (500ms-2s)

- **What happens:** The incident is reported to the cloud control center.
- **Processing:** Multiple cloud services receive the notification simultaneously.
- **Action in parallel:**
 - Public alerts sent to mobile apps.
 - Operator dashboards updated.
 - Emergency vehicle routing calculated.
 - Traffic impact predictions generated.
- **Key point:** Many actions happen concurrently, not sequentially.

Stage 4: Strategic Response (2s onward)

- **What happens:** Broader system adjustments and learning.
- **Processing:** City-wide optimization and pattern analysis.
- **Action:** Long-term traffic management strategies are adjusted.
- **Key point:** The system learns from each incident to improve future responses.

The **four-stage** flow ensures that safety-critical decisions happen immediately at the local level while still enabling comprehensive city-wide coordination and learning.

7.2.4. Communication Infrastructure and Protocols

The architecture uses **multiple communication protocols**, with each protocol selected to meet specific performance, reliability, and security requirements.

○ **Edge-to-Edge Communication**

Primary Protocol: DSRC (Dedicated Short-Range Communications) or C-V2X (Cellular Vehicle-to-Everything): wireless technologies designed for fast, direct communication between nearby devices.

Purpose: Enables immediate coordination between adjacent intersections without relying on cloud systems.

Use Case: Supports synchronized “green wave” traffic signals and coordinated emergency vehicle priority across multiple intersections.

- **Edge-to-Cloud Communication**

Primary Protocol: MQTT (Message Queuing Telemetry Transport) over TLS (Transport Layer Security): a lightweight messaging protocol secured with encryption.

Characteristics: Efficient for devices with limited computing power, reduces network overhead, and supports different levels of delivery reliability.

Quality of Service (QoS) Implementation:

- QoS 0 (At most once): Sensor telemetry data using best-effort delivery.
- QoS 1 (At least once): Configuration updates with guaranteed delivery.
- QoS 2 (Exactly once): Control commands requiring precise and safe delivery for safety-critical operations.

- **Cloud Internal Communication:**

Service-to-Service Communication:

- gRPC (Google Remote Procedure Call): A high-performance communication framework used for time-sensitive service interactions
- REST (Representational State Transfer) APIs: Standard web-based interfaces used for management and administrative functions

Event Distribution:

- Apache Kafka: A distributed event-streaming platform used for reliable data exchange.
- Avro: A compact data serialization format used to efficiently structure and transmit data.

Service Discovery: Kubernetes-native service discovery or a dedicated service registry such as Consul, which helps services automatically find and communicate with each other.

7.2.5. Security Architecture

The security architecture follows a **defense-in-depth strategy**, which means multiple layers of protection are applied across the system to reduce security risks.

- **Perimeter Security**

- A Web Application Firewall (WAF) is used to protect all public-facing APIs from malicious traffic.
- Distributed Denial of Service (DDoS) protection is implemented with automatic traffic filtering to prevent service disruption.
- The API Gateway enforces strict request validation and schema checks to block invalid or malicious requests.

- **Network Security**

- A Zero Trust Network Architecture is applied, requiring authentication for all system communications.
- Micro segmentation separates system components using clearly defined firewall rules to limit lateral movement within the network.
- Virtual Private Network (VPN) tunnels secure all communications between cloud services and edge devices.

- **Application Security**

- All microservices are developed in accordance with OWASP Top 10 security guidelines.
- Regular vulnerability scanning and penetration testing are conducted to identify and address security weaknesses.
- Secure coding practices are integrated into the development pipeline to reduce risks during software development.

- **Data Security**
 - All stored data is protected using AES-256 encryption.
 - All data transmitted across the network is secured using TLS 1.3 encryption.
 - Field-level encryption is applied to particularly sensitive data elements.
 - Hardware Security Modules (HSMs) are used to securely manage cryptographic keys.
- **Identity and Access Management**
 - Multi-factor authentication (MFA) is required for administrative access.
 - Role-Based Access Control (RBAC) is implemented using the principle of least privilege.
 - Comprehensive audit logging records all access attempts and system changes for monitoring and compliance.

7.2.6. Scalability and Performance Design

The system is designed to grow smoothly over time while maintaining strong performance.

- **Scalability Strategy**
 - **Intersection Expansion:** New intersections can be added by installing additional edge gateways without changing the overall architecture.
 - **Data Volume Management:** Apache Kafka can be scaled by adding partitions and consumer groups as data volume increases.
 - **Processing Capacity:** Microservices can scale horizontally by increasing the number of service replicas based on system demand.
 - **Storage Scaling:** Database sharding and read replicas are used to handle growing data storage and access needs.
- **Performance Targets**
 - **Edge Processing Latency:** Less than **100 milliseconds** for local traffic signal decisions.
 - **Event Processing Throughput:** Up to **50,000 events per second** per Kafka broker.
 - **Dashboard Query Response:** Less than **2 seconds** for 95% of user queries.
 - **System Health Monitoring:** Critical component failures detected within **5 seconds**.
- **Capacity Planning**
 - **Initial Development:** 200 intersections, 1,000 sensors, and 50 concurrent users.
 - **Medium Term (3 years):** 500 intersections, 5,000 sensors, and 200 concurrent users.
 - **Long Term (5 years):** More than 2,000 intersections, 20,000+ sensors, and 500+ concurrent users.

7.2.7. Reliability and Fault Tolerance Mechanisms

The architecture ensures continuous operation even when failures occur.

- **Failure Handling**
 - **Cloud Center Failure:** Edge devices continue basic traffic control using cached configurations.
 - **District Layer Failure:** Nearby district coordinators take over responsibilities, and edge devices switch to fallback coordination.
 - **Edge Device Failure:** Traffic controllers revert to pre-programmed signal timing, while nearby intersections adjust automatically.
 - **Network Disconnection:** Edge devices operate independently and synchronize data once the connection is restored.

- **Sensor Failure:** Redundant sensors and estimation models compensate for missing or incorrect data.
- **Disaster Recovery**
 - **Recovery Time Objective (RTO):** Full system recovery within **3 hours**.
 - **Recovery Point Objective (RPO):** Maximum **15 minutes** of data loss.
 - **Backup Strategy:** Real-time replication to a secondary region, hourly incremental backups, and daily full backups stored for 30 days.
 - **Recovery Testing:** Automated failover tests conducted quarterly to ensure recovery plans remain effective.

7.2.8. Implementation of Phasing Approach

The system is deployed in **phases** to reduce risk and deliver value early.

Phase 1: Foundation Setup (Months 1–6)

- Deploy and configure core cloud infrastructure.
- Develop the central dashboard and administrative tools.
- Select 20 pilot intersections with different traffic conditions.
- Train operators and define operational workflows.

Phase 2: Pilot Deployment and Evaluation (Months 7–12)

- Deploy the system to the 20 pilot intersections.
- Conduct a three-month performance evaluation and collect stakeholder feedback.
- Improve the system based on real-world usage.
- Plan for large-scale deployment.

Phase 3: Scaled Deployment (Months 13–24)

- Expand to the remaining 180 intersections at a rate of 30–40 per quarter.
- Integrate with other city systems and external data sources.
- Enhance AI capabilities using accumulated data.
- Perform system-wide performance optimization and reporting.

7.2.9. Strategic Advantages and Design Rationale

This hybrid architecture provides several key advantages:

- **Balanced Decision-Making:** Fast decisions at the edge, coordinated control at the district level, and long-term optimization in the cloud.
- **Progressive Enhancement:** The system delivers value gradually as components are added, without requiring full deployment upfront.
- **Future-Proof Design:** Modular structure supports future technologies such as autonomous vehicles, dynamic pricing, and advanced analytics.
- **Risk-Managed Deployment:** Phased rollout allows learning, adjustment, and risk reduction.
- **Built-in Compliance:** Privacy, security, and auditing are included from the start rather than added later.

7.3. Summary and Strategic Rationale

The proposed **hybrid edge–cloud architecture** offers a strong balance between innovation and practicality for the Smart City Traffic Management System. By distributing intelligence across edge, district, and cloud layers, the system meets strict performance requirements while remaining flexible and scalable for long-term growth.

This architecture directly addresses the four challenges identified in Section 2.1:

- **Real-time response issues:** Solved through edge autonomy.
- **Legacy system limitations:** Managed through adapters and protocol translation.
- **Data silos:** Eliminated using centralized event streaming and shared data models.
- **Performance bottlenecks:** Avoided through horizontal scaling and efficient data processing.

Economic analysis shows a strong return on investment, with cost recovery expected within **12–18 months**. The security-first approach ensures regulatory compliance and infrastructure protection. The phased deployment strategy minimizes risk while allowing continuous improvement.

Overall, this architecture is not only a technical solution but a **long-term strategic foundation** for the city's smart transportation ecosystem. It delivers immediate improvements in traffic flow, safety, and environmental impact, while providing a flexible platform that can evolve with future mobility technologies.

8. SRED Risk Analysis

This section focuses on the risks that could affect the requirements process itself for the Smart City Traffic Management System (SCTMS). The requirements stage is critical because mistakes here can cause problems later, such as building the wrong features or going over budget. We identify six major risks and give practical plans to prevent or reduce them. These risks are common in systems projects that involve many stakeholders and complex technology, like real-time traffic systems. Managing them early helps keep the project on track and ensures the final system meets real needs.

8.1. Risk 1: Changing Stakeholder Requirements (Requirements Volatility)

A common risk is that stakeholders change their minds about what they want after the requirements have been agreed. For example, city officials might add new rules due to change in local policy, or traffic operators might realize during testing that they need extra dashboard features. This can lead to extra work, delays, and higher costs if it's not handled well. (Geeksforgeeks, 2025)

To reduce the risk, the project should use iterative requirements process with regular review meetings every two to four weeks. All key stakeholders would see updated requirement documents and give feedback early. A formal change control process is also needed. Any new or changed requirement must be written down, assessed for impact on time and cost, and approved by a small review group (including representatives from operations, IT, and management). This way, changes are allowed but controlled, preventing uncontrolled growth. (Clockwise Software, 2024)

8.2. Risk 2: Conflicting Requirements from Different Stakeholder Groups

One of the biggest challenges in a project like SCTMS is that different stakeholder groups often want different things that can clash with each other. For instance, traffic control operators might prioritize a simple, fast dashboard with large buttons for quick overrides during busy shifts, while government officials might want detailed analytics and complex reports for policy decisions. Emergency services could demand absolute priority for their vehicles in all situations, which might slow down regular traffic flow and frustrate road users. If these conflicts are not spotted and resolved early, the final requirements could favor one group too much, leading to a system that some users resist or find hard to use.

This kind of conflict is common in multi-stakeholder projects because each group sees the system from their own viewpoint, such as operators focusing on daily operations, IT engineers on technical limits, and the public on smoother commutes. Without careful handling, these differences can cause arguments later, rework, or even lower acceptance of the finished system.

To manage this risk, the team should actively look for conflicts during the elicitation phase. One effective way is to hold joint prioritization workshops where representatives from all groups (as listed in Section 3) discuss and rank requirements together. Use techniques like voting or trade-off discussions to reach agreement. For instance, decide if emergency priority always overrides congestion optimization, or only in certain cases. Document all decisions and the reasons behind them, so everyone understands the compromises. A neutral facilitator can help keep discussions fair. Finally, include conflict checks in the requirements review process: before final approval, go through the list specifically asking if the requirement causes issues for any other group. This early resolution keeps the requirements balanced and reduces surprises later. (LinkedIn, 2025)

8.3. Risk 3: Missing Hidden or Unstated Requirements

Another important risk in the requirements process is overlooking requirements that are not clearly expressed or immediately obvious at the start. These are often called "hidden" or "unstated" requirements. For example, stakeholders might assume the system will automatically switch to a safe backup mode during a major power outage or network failure, but they never mention it because it seems normal to them. Other examples include how the system should handle rare but disruptive events, such as long-term roadworks that block lanes for weeks, extreme weather causing sudden low visibility, or a large public event that brings thousands of extra vehicles into the city. There could also be less visible needs, like how the system logs and recovers from partial sensor failures without losing critical data, or how it behaves when multiple emergencies happen at the same time. If these are missed, the system might perform well in everyday situations but fail or behave unpredictably when unusual conditions occur, leading to safety issues, loss of trust, or expensive fixes later in development.

This risk is especially high in complex projects like SCTMS because the system operates in a real-world environment with many unpredictable factors. Stakeholders from different groups (operators, emergency services, IT engineers) often focus on their main daily concerns and may not think about edge cases or failure scenarios until much later. Road users, for instance, might not mention needs related to getting accurate alerts during blackouts because they expect the system to "just work." Without active effort to uncover these, the final requirements document can look complete on paper but leave important gaps that only show up during testing or actual use.

To reduce this risk, the project team should combine several structured techniques. As planned in Section 3, use scenario-based interviews and "what-if" questions. For instance, asking emergency services how the system should handle two ambulances needing priority in opposite directions, or asking operators about dashboard behavior if sensors go offline in a storm. Create use cases covering normal operation, edge cases, and failure/recovery modes. Introduce early prototypes or simple mock-ups to help stakeholders spot gaps more easily, as seeing something tangible often brings out new ideas. Build a review checklist with categories like error handling, recovery, stress performance, and environmental disruptions, and go through it with all stakeholder groups. Finally, cross-check against industry standards or similar traffic projects to catch common oversights. This combined approach helps capture a fuller set of requirements, making the system safer and more reliable in real-world use. (Geeksforgeeks, 2025)

8.4. Risk 4: Scope Creep

Scope creep is a common risk where new ideas or features gradually get added to the project without proper control, causing it to grow beyond the original plan. For example, during requirements discussions or workshops, a stakeholder might suggest including features like parking space monitoring, real-time public bus tracking, or integration with street lighting systems. Even though these were clearly marked as out-of-scope in Section 2.2. While these ideas might sound useful, adding them without careful review can increase development time, raise costs, and delay the whole project. In the worst case, it can push back the launch date or force the team to cut important core features to stay on budget.

This risk often happens slowly and with good intentions. People see opportunities to make the system better and don't realize the impact of small additions. In traffic management projects, where many groups are involved, it's easy for "just one more thing" requests to build up and blur the original focus on traffic flow optimization.

Mitigation starts with a clear and visible scope document shared at every meeting. Use prioritization methods like MoSCoW (Must have, Should have, Could have, Won't have) to rank requirements from the beginning. Once the baseline requirements are approved, any suggested addition goes through the change control process mentioned in Risk 1, with clear estimates of extra time and cost. Regular progress reports to stakeholders highlighting how new requests affect the schedule will encourage discipline.

8.5. Risk 5: Technical Feasibility Misjudgment with Legacy Systems

A specific risk for this project is underestimating how hard it will be to connect new components to older traffic infrastructure. Requirements might assume smooth integration using standard APIs, but legacy controllers could have undocumented limits or unusual data formats, causing problems later in development.

The plan is to involve IT and integration engineers from the very start (as in Section 3.4) and conduct early technical assessments. Run small proof-of-concept tests with real or simulated legacy equipment to check compatibility. Document all assumptions about legacy behavior and review them in joint sessions. If gaps appear, adjust requirements early or add specific integration requirements to handle the issues. This front-loaded effort saves larger rework later.

8.6. Risk 6: Incomplete or Biased Stakeholder Involvement

Not all stakeholder groups might participate fully. For example, road users (drivers) could give low survey responses, or emergency services might be too busy for deep interviews. This can lead to requirements that favor one group over others, reducing overall acceptance of the system.

To address this, set participation targets for each group and follow up actively (e.g., multiple survey reminders, flexible interview times). Offer different ways to give input, such as short online forms for busy emergency teams or public online consultations for drivers. Track involvement levels and, if a group is under-represented, arrange extra targeted sessions. A final validation workshop with representatives from all groups helps confirm balanced coverage.

8.7. Summary

In summary, the requirements process for the Smart City Traffic Management System (SCTMS) faces several common but serious risks that could affect the project's success. The six risks discussed by changing stakeholder needs, conflicting requirements between groups, missing hidden or unstated needs, scope creep, technical misjudgments with legacy systems, and incomplete stakeholder involvement are typical in projects with multiple stakeholders and complex real-time technology. If not managed well, these risks can lead to unclear requirements, extra costs, delays, or a final system that does not fully meet user expectations or work reliably in real conditions.

By putting practical plans in place early, such as iterative reviews, change control processes, joint workshops, scenario-based questions, early prototypes, clear scope boundaries, technical proofs-of-concept, and active follow-up for participation, the project team can greatly reduce these dangers. These steps help create a more accurate, balanced, and complete set of requirements while keeping the project focused and on schedule. Regular risk checks throughout the elicitation and review phases will allow the team to spot new issues quickly and adjust plans as needed. Overall, this proactive

approach increases the chances of delivering a traffic management system that is effective, accepted by all stakeholders, and able to handle the challenges of a modern urban environment.

References

- 29148, I. (2014). *IEEE SA*. Retrieved from <https://standards.ieee.org/ieee/29148/6937/>
- Berkeley. (2001). Retrieved from Freeway Performance Measurement System: <https://pems.dot.ca.gov/>
- Clockwise Software. (2024, February 5). *Clockwise Software*. Retrieved January 7, 2026, from Clockwise Software: <https://clockwise.software/blog/software-development-risks>
- Geeksforgeeks. (2025, July 11). *Geeksforgeeks*. Retrieved January 7, 2026, from Geeksforgeeks: <https://www.geeksforgeeks.org/software-engineering/software-engineering-challenges-eliciting-requirements>
- government, N. (2022). *SCATS*. Retrieved from <https://www.scats.nsw.gov.au/organisation/about-us>
- Leotek. (2025). *Leotek*. Retrieved December 30, 2025, from <https://leotek.com/laws-and-regulations-of-smart-traffic/>
- LinkedIn. (2025). *LinkedIn*. Retrieved January 7, 2026, from LinkedIn: <https://www.linkedin.com/advice/3/what-risks-do-you-face-when-eliciting-requirements>
- McBride, K. (2025). *Miovision*. Retrieved December 30, 2025, from <https://miovision.com/blog/what-is-smart-traffic-management/>
- Meadowlands Adaptive Signal System for Traffic Reduction (MASSTR). (2023). *Meadowlands Adaptive Signal System for Traffic Reduction (MASSTR)*. Retrieved from <https://www.njsea.com/transportation/masstr/>
- Mehta, P. (2025). *Zealous System*. Retrieved December 30, 2025, from <https://www.zealousys.com/blog/integrating-iot-sensors-ai-for-smarter-traffic-monitoring/>
- Milhouse. (2025). *Milhouse*. Retrieved December 30, 2025, from <https://milhouseinc.com/smart-traffic-management-systems-improving-conditions-for-drivers-and-cities/>
- Rush, D. (2025, October 20). *DIGI*. Retrieved December 30, 2025, from <https://www.digi.com/blog/post/smart-city-traffic-management-solutions>
- Skladzinski, J. (2025, July 15). *Omnisight*. Retrieved December 30, 2025, from <https://omnisightusa.com/blog/smart-city-traffic-management-complete-guide?>
- Sommerville. (2016). *Pearson*. Retrieved from <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000006356>
- Sommerville, I. (2016). Software Requirements Engineering. In I. Sommerville, *Software Engineering* (pp. 85–120). Harlow, England: Pearson Education.
- Systems, D. o.-I. (n.d.). Retrieved from Intelligent Transportation Systems Joint Program Office: <https://www.its.dot.gov/>
- Wieger, K. (2013). Retrieved from <https://www.oreilly.com/library/view/software-requirements-3rd/9780735679658/>
- Wiegers, K., & Beatty, J. (2013). Software Requirements. In K. Wiegers, & J. Beatty, *Software Requirements*. Redmond, WA: Microsoft Press.